Why a completed payment form is not enough

Payment authorization XSS CSRF and a security lab I built

Dhruba Poudel | 28 October 2026

View source code | Download the lab | Read online

In 2018, Facebook acknowledged me in its White Hat researcher list for responsible vulnerability disclosure. I remember reporting an advertising payment issue in which ads could continue running for days without completed credit-card entry. That description is my recollection; the acknowledgement confirms the recognition. I built the fictional lab below to explain the engineering lessons. It is original demonstration code, not Facebook source code or a reconstruction of my report.

The question I keep coming back to is straightforward: what must the server know before a campaign is allowed to run? A completed form does not establish account authority, a valid billing arrangement or permission to spend. I built a small campaign service to make those decisions visible, then added regression tests for repeat requests, queued work and browser trust boundaries. Here is how it works, where the controls belong and what the tests demonstrate.

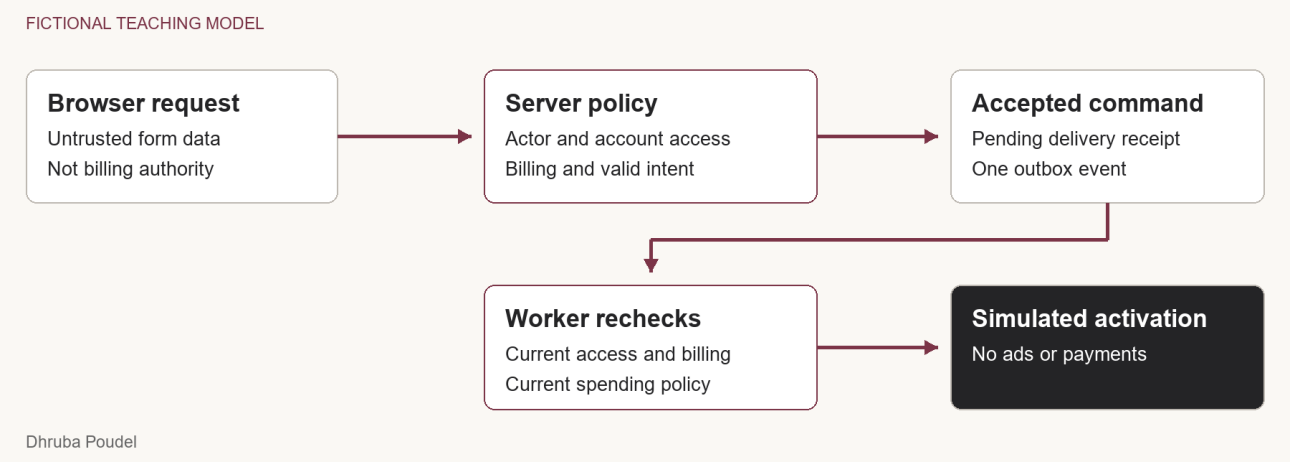
Define the business invariant before the interface

I gave the fictional service accounts, campaigns and an activation command. An account operator can request activation for a campaign owned by that account. An accepted command moves the campaign to PENDING_DELIVERY; a separate simulated worker can later make it ACTIVE. Before either transition, I check the conditions that matter: current authority, the account and campaign relationship, lifecycle, spending intent and trusted billing eligibility. Those conditions define the invariant the code must preserve.

I started with a deliberately flawed function. It treats a browser flag as authority to activate a campaign. The flag may be useful for deciding which screen to show, but it says nothing about whether the account has an approved payment arrangement. In this function, setting paymentStepComplete to true selects ACTIVE even when no billing evidence exists. The small example makes the trust error easy to see.

The deliberately flawed decision from campaign-security.mjs

```
export function vulnerableDecision(clientForm) {
  return clientForm.paymentStepComplete === true ? 'ACTIVE' : 'DRAFT';
}
```



An original fictional workflow separates request acceptance from the later delivery decision

I check client-controlled values by asking what happens when the normal screen is absent. Would the execution path still reject the command? If the answer depends on a disabled button, a required input or a hidden field, the decision is in the wrong place. I want the interface and server to agree, but the server must enforce the rule even when the interface is bypassed.

In the corrected model, I ignore paymentStepComplete and billingVerified inside the request and retrieve the account's billing record from the store. I allow either verified card billing or verified approved credit. This distinction matters: an account without a card can still have a legitimate arrangement. A product might support invoicing, prepaid funding or another approved model. I want the control to enforce the business policy, not accidentally turn one payment-screen design into a universal rule.

Trusted billing eligibility in the fictional policy

```
export function billingEligible(account) {
  return account.billing?.status === 'VERIFIED' &&
    ['CARD', 'APPROVED_CREDIT'].includes(account.billing.kind);
}
```

Bind the actor account and campaign together

I treat actor as an identity supplied by a trusted server session, never as a value chosen in the request body. Authentication itself sits outside this lab. Inside the activation path, I check that the account exists, is active and includes the actor in its operator list. I then verify that the selected campaign belongs to that account. A valid user and a valid campaign are not enough unless the relationship between them is valid too.

Authorization method excerpt from CampaignStore

```
#authorize(actor, accountId, campaignId) {
    // actor is obtained from an authenticated server session, never a form field.
    const account = this.#accounts.get(accountId);
    const campaign = this.#campaigns.get(campaignId);
    if (!account || account.status !== 'ACTIVE' || !account.operators.includes(actor))
      deny('FORBIDDEN');
    if (!campaign || campaign.accountId !== account.id) deny('FORBIDDEN');
    return { account, campaign };
  }
```

I keep these questions distinct. Authentication identifies the person; account authorization decides whether that person may operate the account. The campaign relationship prevents someone from using an account they control to reach another account's object. I return FORBIDDEN for missing or mismatched objects, while the regression cases check each condition separately. For a real service, I would extend that boundary with the required permissions and an audit record that explains accepted and denied decisions.

I added tests for another actor attempting activation, a campaign selected through the wrong account, an unknown account, a suspended account and a forged identity placed inside the request. That last case checks an easy mistake: copying convenient fields from a request into the authorization context. The test confirms that extra actor and account fields in the body do not change the trusted identity used by the store.

Treat money and lifecycle as validated intent

I validate the spending intent before accepting the transition. The budget must be a positive safe integer in cents, remain within the account's per-campaign limit and use the account's currency. I reject numeric strings, fractional cents, NaN, Infinity and unsafe integers. This avoids having different parts of the service interpret the same amount differently. I also test the smallest permitted amount and the exact upper limit so the valid boundaries stay usable.

For this AUD example, integer cents make the rule explicit. A production financial model needs currency-specific minor units, rounding and conversion rules. It also needs to distinguish a per-campaign cap from account-wide funds. My lab checks the former; it does not reserve credit across campaigns, calculate tax

or settle transactions. If several campaigns share a balance, I would model and lock that shared resource rather than assuming that individually valid budgets imply a valid total.

I enforce lifecycle alongside money. Only a DRAFT campaign can enter PENDING_DELIVERY through a new command. Valid billing does not make a cancelled or otherwise ineligible campaign activatable. I check this in the execution path so a caller cannot skip a required transition by presenting a plausible final state. In a larger product, I would make the permitted transitions explicit and test every entry point that can change them.

Make repeat requests refer to the same exact intent

I designed retries around an exact command. After a timeout, a caller may need to repeat a request without knowing whether it was accepted. I scope the receipt by actor, account and idempotency key, then fingerprint the actor, account, campaign, budget and currency. The receipt key locates the previous command. The fingerprint confirms that the repeated command still means the same thing. Reusing a key should not become permission to spend a different amount.

Receipt lookup and exact intent comparison from activate

```
const receiptKey = JSON.stringify([actor, accountId, intent.idempotencyKey]);
    const fingerprint = digest([actor, accountId, campaignId, intent.budgetCents,
      intent.currency]);
    const existing = this.#receipts.get(receiptKey);
    if (existing) {
      if (existing.fingerprint !== fingerprint) deny('IDEMPOTENCY_CONFLICT');
      return copy(existing.result);
    }
```

For an identical retry, I return a copy of the original acceptance receipt. A changed budget or another campaign produces IDEMPOTENCY_CONFLICT. Returning copies also stops a caller from modifying the stored receipt through a shared object reference. I keep authorization and input validation ahead of the receipt lookup: removing someone from the account prevents them from retrieving an earlier result through this interface. The tests cover each of these behaviors.

I use the receipt to describe acceptance, not to prove delivery or grant fresh permission. If billing is revoked after acceptance, the old record may still exist, but the worker must check current policy before activation. A production service also needs a retention period, durable storage and clear behavior when a caller retries after a receipt expires. Those choices determine how useful replay protection remains over time; a key alone does not solve them.

Commit acceptance and dispatch intent together

On acceptance, I update the campaign, create an outbox event and record the receipt together. In this lab those mutations are synchronous, in memory, with no await or external call between them. One Node.js event loop executes them without yielding. That lets me test the relationship between the three records, but it is not a database transaction or crash-recovery mechanism. Restarting the process loses the store.

For a production implementation, I would put the campaign transition, receipt insertion and outbox insertion in one database transaction. I would use unique constraints for receipt and event identities, then row locking or optimistic concurrency for competing updates. The dispatcher needs its own retry policy and downstream deduplication. An external timeout can happen after a side effect occurred, so an internal delivered flag does not establish exactly-once effects across service boundaries.

I made the simulated worker recheck account membership, billing, lifecycle, budget and currency immediately before activation. It uses an existing server-created event rather than trusting a new client assertion that approval happened. The tests revoke billing after acceptance and confirm that the campaign remains pending. They also remove the operator and lower the spending limit. In each case, the earlier accepted command cannot override the current policy.

Current-policy checks before simulated delivery

```
// A queued job must recheck current authority, billing and lifecycle.
    const { account, campaign } = this.#authorize(event.actor, event.accountId,
      event.campaignId);
    if (!billingEligible(account)) deny('BILLING_REQUIRED');
    if (campaign.status !== 'PENDING_DELIVERY') deny('INVALID_TRANSITION');
    if (!Number.isSafeInteger(campaign.budgetCents) || campaign.budgetCents < 1 ||
        campaign.budgetCents > account.maxCampaignCents || campaign.currency !==
          account.currency) deny('INVALID_INTENT');
```

For an already DELIVERED event, I return its existing record on replay. That records what happened; it does not undo an effect or authorize continued service. In a running advertising platform, I would add ongoing eligibility checks, suspension behavior and reconciliation after activation. My worker tests the pending-to-active boundary. Keeping that boundary clear makes it easier to decide which controls need to run once, which must run again and which must remain active throughout delivery.

Reproduce the decisions locally

I ran the demo with two versions of the same command. First I supplied client completion flags without trusted billing. Then I supplied the command to an account with verified approved credit. The output separates the flawed decision, server acceptance and simulated dispatch. The example stays local: it makes no network requests, runs no ads and processes no payment. These are fictional records, so the decisions can be reproduced without touching a real service.

Observed output from node demo.mjs

```
Fictional local teaching model. No Facebook or payment service is contacted.
Flawed UI-completion decision: ACTIVE
Server decision with no trusted billing: BILLING_REQUIRED
Server decision with approved account credit: PENDING_DELIVERY
Repeated command returns same event: true
Revalidated simulated dispatch: DELIVERED
Final fictional campaign state: ACTIVE
```

I check the denied path as carefully as the successful one. A rejected command must leave no activation event behind. My regression test therefore checks the error and the resulting state: the campaign stays DRAFT and the outbox stays empty. This catches code that displays a failure after already changing a record or enqueueing work. The caller’s error message is only one part of the outcome.

Regression test checking denial and unchanged state

```
test('missing server billing cannot be bypassed by client flags', () => {
  const store = makeStore({ billing: null });
  assert.throws(() => activate(store, { request: request({ paymentStepComplete: true,
    billingVerified: true }) }), code('BILLING_REQUIRED'));
  assert.equal(store.snapshot().outbox.length, 0);
  assert.equal(store.snapshot().campaigns[0].status, 'DRAFT');
});
```

Keep XSS separate from the payment policy

I extended the review to XSS because campaign labels, comments and imported text can cross a different trust boundary. When I put untrusted content into innerHTML, the browser interprets it as markup. For a plain label, I want text, not markup interpretation. I keep this review separate from the billing rule: authorization decides whether an action is permitted, while output handling decides how supplied content reaches the browser.

For a fixed ordinary div displaying a label, I use textContent. The helper below keeps the value in a text context. I would not apply that advice blindly to a script element, style element, URL or event handler; those are different destinations with different risks. If a feature genuinely requires rich HTML, I would define an

allowed-content policy and use an appropriate maintained sanitizer instead of trying to remove a few suspicious strings.

The actual text-only rendering helper

```
export function renderLabel(element, text) {
  element.textContent = String(text);
}
```

I tested the helper with a harmless markup-shaped marker and a fake element whose innerHTML setter throws. The test verifies that the string is assigned to textContent and the HTML sink is not used. It is a unit test, not browser execution or sanitizer coverage. In a real interface, I would follow the value from its source to each destination and add browser tests for the actual rendering contexts.

In React, I use normal text interpolation when a value is meant to be text. I still review raw HTML insertion, URL construction and third-party components separately. A restrictive Content Security Policy can provide another layer, but it cannot replace correct output handling. I would test the actual policy against the application's behavior rather than treating the presence of a CSP header as proof that XSS is prevented. The local lab does not implement a CSP.

Distinguish CSS styling from CSRF protection

CSS stands for Cascading Style Sheets; CSRF is cross-site request forgery. I separate presentation from request security. Hiding a button with CSS cannot revoke server permission, and a green success badge cannot establish billing eligibility. For configurable labels or themes, I prefer a small enum such as warning or success, mapped to classes the application owns. That gives the user a visual choice without accepting arbitrary stylesheet text.

Conceptual styling example not included in the executed lab tests

```
// label is an application-owned ordinary div.
const variants = Object.freeze({
  warning: 'label--warning',
  success: 'label--success'
});
const selected = typeof userChoice === 'string' &&
  Object.hasOwn(variants, userChoice) ? userChoice : 'warning';
label.className = `label ${variants[selected]}`;
```

This is a conceptual styling example rather than part of the executed tests. The user selects one of two variants and the application owns both class definitions. The choice changes appearance only; it does not create HTML, grant permission or authorize a campaign.

For CSRF, I start with the browser's authenticated request. If a browser automatically includes a session credential, the server needs controls against an unwanted cross-site state change. I added a standalone helper that checks the exact configured origin and compares a supplied token with a session-bound value. This browser-only example rejects a missing origin. A real integration must define how trusted proxies, legitimate clients and non-browser requests are handled.

Standalone CSRF verification helper from the lab

```
export function requireCsrf({ origin, expectedOrigin, token, sessionToken }) {
  // expectedOrigin and sessionToken come from trusted server configuration/session.
  if (!expectedOrigin || origin !== expectedOrigin) deny('CSRF_ORIGIN');
  if (typeof token !== 'string' || typeof sessionToken !== 'string' ||
      token.length < 32 || token.length > 256 || sessionToken.length > 256)
        deny('CSRF_TOKEN');
  const supplied = Buffer.from(token, 'utf8');
  const expected = Buffer.from(sessionToken, 'utf8');
  if (supplied.length !== expected.length || !timingSafeEqual(supplied, expected))
    deny('CSRF_TOKEN');
}
```

I treat expectedOrigin and sessionToken as trusted server values. A real session system must generate unpredictable tokens, bind them to the correct session and deliver them safely. The fixed value in my tests is a fixture. The helper has no HTTP handler, cookie setup or token issuance, so I would integrate and test those parts before using it in an application. A correct comparison is one control inside the request pipeline.

I do not treat a valid token as proof that someone understood or consented to a particular budget. For a sensitive action, I would bind the confirmation to the server-validated intent and still check account authority. XSS prevention also remains necessary: a script executing in the trusted origin can undermine browser-based request protections. Each control needs a clear job. CSRF checks do not replace authorization, and they do not repair an XSS flaw.

Apply the same distinctions to WordPress development

I apply the same distinctions to WordPress themes, plugins and API handlers. A nonce helps protect an action against CSRF, but it is not authentication, authorization or one-time replay protection. A valid nonce does not mean the user can modify a particular record. I still need capability checks such as current_user_can and the relevant object-level permissions. I review those checks at the handler, where the state change occurs.

For a WooCommerce integration, I trace the whole route: authentication context, capability, object ownership, validated input, lifecycle, side effect and output rendering. I also check which records establish payment status and how provider events become trusted. A good admin screen cannot compensate for an unsafe backend decision. A correct backend decision does not prevent an unrelated rendering flaw. I test the paths separately so a pass in one area does not hide a gap in another.

Report what the tests actually prove

I ran all 41 automated tests successfully on 28 October 2026. They cover denial paths, valid transitions and selected boundary cases. The replay test schedules 25 callers with promises, but their store mutations run synchronously in one Node.js event loop. That checks repeated-command behavior in this model. It does not test multiple database workers, load capacity, crash recovery or distributed concurrency, which need different infrastructure and failure cases.

Test area	Representative observed checks	Boundary
Billing and ownership	Client billing flags cannot replace trusted billing; approved credit succeeds; cross-account commands fail.	Fictional server records, not a payment provider.
Intent and lifecycle	Invalid cents, mismatched currency, non-draft state and malformed keys fail; valid budget boundaries pass.	Per-campaign limits, not aggregate credit reservation.
Replay and delivery	Same intent returns one event; changed intent conflicts; pending jobs respect membership, billing and limit revocation.	Single synchronous in-memory store.
Browser helpers	Text-only sink selected; configured origin and matching token accepted; missing, cross-origin or invalid inputs rejected.	Fake DOM and standalone helper, not full browser or HTTP integration.

My next engineering step would be transactional storage, competing worker tests, crash recovery and browser-level rendering checks. Those extensions are proposed, not completed. For now, the package gives a reader a traceable path from input to decision to state change. It also gives me a regression baseline: when I

add persistence or another billing model, I can check that the existing denied cases remain denied and legitimate commands still work.

Assess impact against the demonstrated conditions

I assess impact from the demonstrated conditions. Here, the flawed function selects ACTIVE from a client flag without trusted billing evidence. For a real report, I would record starting privileges, account scope, actual side effects, legitimate credit arrangements and the controls that limit exposure. I separate observed results from untested consequences. The goal is a report that explains the risk precisely enough for another engineer to reproduce the decision and repair the right boundary.

When I review a workflow, I want to answer one concrete question: can an accepted command create a state that contradicts the account's current authority or policy? I built this lab to make that question testable. The checks belong where the action happens, and the tests must inspect the resulting state. That is the standard I want to bring to payment flows, WordPress integrations and operational approval systems.